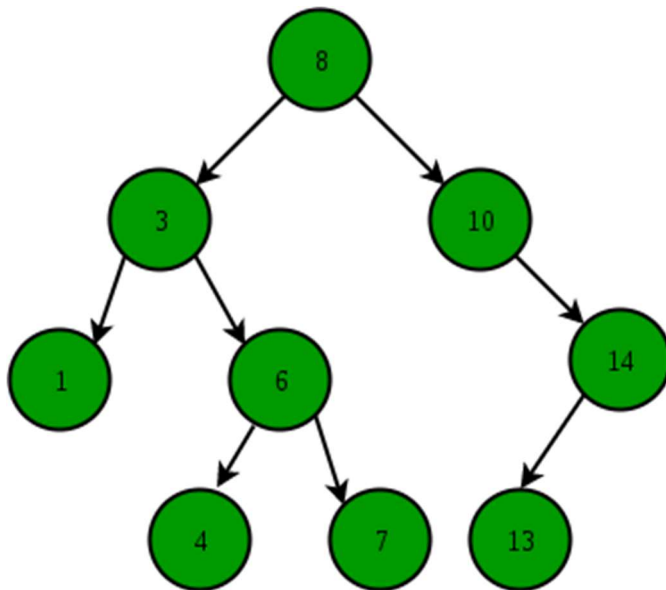


Binary Search Tree | Set 1 (Search and Insertion)

Binary Search Tree, is a node-based binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.
There must be no duplicate nodes.



The above properties of Binary Search Tree provide an ordering among keys so that the operations like search, minimum and maximum can be done fast. If there is no ordering, then we may have to compare every key to search a given key.

Searching a key

To search a given key in Binary Search Tree, we first compare it with root, if the key is present at root, we return root. If key is greater than root's key, we recur for right subtree of root node. Otherwise we recur for left subtree.

```
// C function to search a given key in a given BST
struct node* search(struct node* root, int key)
{
    // Base Cases: root is null or key is present at root
    if (root == NULL || root->key == key)
        return root;
}
```

```

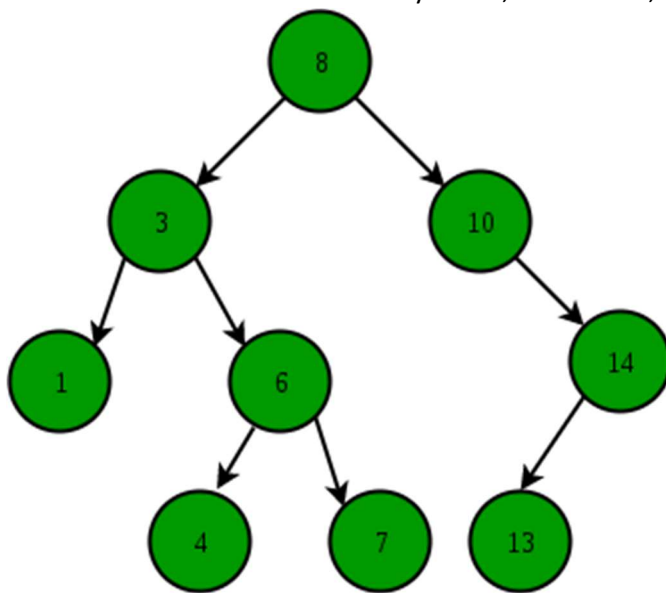
// Key is greater than root's key
if (root->key < key)
    return search(root->right, key);

// Key is smaller than root's key
return search(root->left, key);
}

```

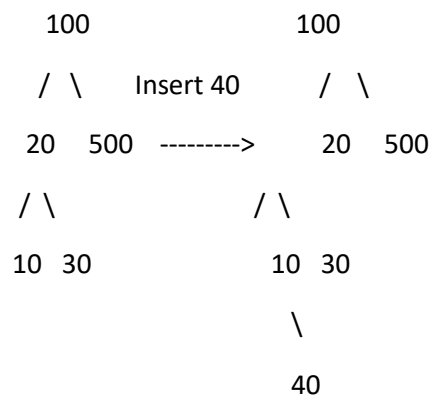
Illustration to search 6 in below tree:

1. Start from root.
2. Compare the inserting element with root, if less than root, then recurse for left, else recurse for right.
3. If element to search is found anywhere, return true, else return false.



Insertion of a key

A new key is always inserted at leaf. We start searching a key from root till we hit a leaf node. Once a leaf node is found, the new node is added as a child of the leaf node.



// C program to demonstrate insert operation in binary search tree

#include<stdio.h>

#include<stdlib.h>

struct node

{

int key;

*struct node *left, *right;*

};

// A utility function to create a new BST node

*struct node *newNode(int item)*

{

*struct node *temp = (struct node *)malloc(sizeof(struct node));*

temp->key = item;

temp->left = temp->right = NULL;

return temp;

}

// A utility function to do inorder traversal of BST

*void inorder(struct node *root)*

{

if (root != NULL)

{

inorder(root->left);

printf("%d \n", root->key);

inorder(root->right);

}

}

/ A utility function to insert a new node with given key in BST */*

struct node insert(struct node* node, int key)*

{

/ If the tree is empty, return a new node */*

if (node == NULL) return newNode(key);

/ Otherwise, recur down the tree */*

if (key < node->key)

node->left = insert(node->left, key);

else if (key > node->key)

node->right = insert(node->right, key);

/ return the (unchanged) node pointer */*

return node;

}

// Driver Program to test above functions

int main()

{

/ Let us create following BST*

```

      50
     /  \
    30   70
   / \  / \
  20 40 60 80 */
struct node *root = NULL;
root = insert(root, 50);
insert(root, 30);
insert(root, 20);
insert(root, 40);
insert(root, 70);
insert(root, 60);
insert(root, 80);

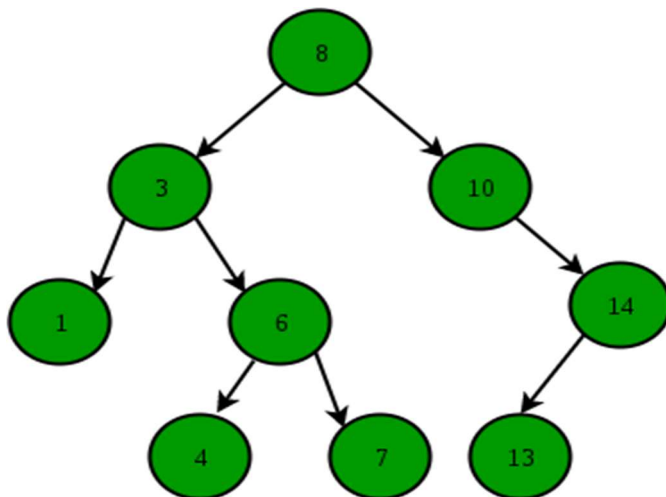
// print inorder traversal of the BST
inorder(root);

return 0;
}

```

Illustration to insert 2 in below tree:

1. Start from root.
2. Compare the inserting element with root, if less than root, then recurse for left, else recurse for right.
3. After reaching end, just insert that node at left(if less than current) else right.



Time Complexity: The worst case time complexity of search and insert operations is $O(h)$ where h is height of Binary Search Tree. In worst case, we may have to travel from root to the deepest leaf node. The height of a skewed tree may become n and the time complexity of search and insert operation may become $O(n)$.